

# Praktikum 2 — Data Acquisition and Wrangling

**Data Science • Magister Ilmu Komputer, FMIPA UGM** Companion to **Chapter 2**. Assignment weight: **3%**.

## What you will do

1. Read a messy CSV wrongly, then correctly — and measure the damage.
2. Profile the data before touching it.
3. Remove duplicates, fix types, enforce validity rules.
4. Diagnose the missingness mechanism (MCAR / MAR / MNAR) and treat it.
5. Detect outliers three ways and decide what to do with them.
6. Join a reference table *with the cardinality checked*.
7. Reshape into tidy form and save a model-ready file.

**The dataset.** `data/sales_raw.csv` is a retail extract exported from an imaginary ERP system. Every defect in it occurs routinely in real exports.

```
import sys
from pathlib import Path

ROOT = Path.cwd()
while not (ROOT / "checks.py").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))

import checks
import numpy as np
import pandas as pd

DATA = ROOT / "data"
pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 30)
```

## 1. Reading it wrongly (DEMO)

First, the naive read — the one everybody does on the first attempt.

```
naive = pd.read_csv(DATA / "sales_raw.csv")
print("shape:", naive.shape)
naive.head(3)
```

One column. The file is **semicolon**-delimited, so `pandas` put every record into a single string column. That failure at least is loud.

Now the subtler one — get the separator right and stop there.

```
half_right = pd.read_csv(DATA / "sales_raw.csv", sep=";")
print("shape:", half_right.shape)
print(half_right.dtypes.to_string())
half_right.head(3)
```

This *looks* fine, and it is thoroughly broken. Four defects, none of which raised an error:

```

print("1. store_id lost its leading zeros:")
print("    ", half_right["store_id"].head(3).tolist(), "<- should be '00042'-style\n")

print("2. unit_price is a string, because of European number formatting:")
print("    ", half_right["unit_price"].dropna().head(3).tolist(), "\n")

print("3. quantity contains a sentinel, not data:")
print("    ", sorted(half_right["quantity"].unique())[:4], "\n")

print("4. order_date is a string, and it is ambiguous:")
print("    ", half_right["order_date"].head(3).tolist(),
      "<- dd/mm or mm/dd?")

```

What the sentinel does to a summary statistic, if you never look:

```

print(f"mean quantity as read    : {half_right['quantity'].mean():.3f}")
print(f"mean quantity, sentinel removed: "
      f"{half_right.loc[half_right['quantity'] > 0, 'quantity'].mean():.3f}")

```

## 2. Reading it correctly (DEMO)

Every argument below exists to defuse one of the defects above.

```

SENTINELS = ["", "NA", "N/A", "-", "null", "NULL", "unknown", "-999"]

raw = pd.read_csv(
    DATA / "sales_raw.csv",
    sep=";",
    dtype={"store_id": "string"},
    na_values=SENTINELS,
    keep_default_na=True,
    thousands=".",
    decimal=",",
    parse_dates=["order_date"],
    dayfirst=True,
)
print("shape:", raw.shape)
print(raw.dtypes.to_string())
raw.head(3)

```

Verifying the date parse actually did what we asked. The trick: find a row whose day-of-month exceeds 12 — under the wrong interpretation it could not have parsed at all.

```

print("max day-of-month observed:", raw["order_date"].dt.day.max())
print("date range:", raw["order_date"].min().date(), "to",
      raw["order_date"].max().date())
print("\nIf the max day is > 12 and the range is sensible, dayfirst was right.")

```

### TASK 2.1

How many **distinct orders** does the raw file contain? (The file has more rows than orders — that is the next section's problem.)

```

n_order_unik = None      # a number, or a DataFrame/Series whose len() is the answer

checks.task("2.1", n_order_unik)

```

### 3. Profiling before cleaning (DEMO)

Characterise the damage before repairing anything.

```
def profile(df):
    out = pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "n_missing": df.isna().sum(),
        "pct_missing": (df.isna().mean() * 100).round(1),
        "n_unique": df.nunique(dropna=True),
        "example": [df[c].dropna().iloc[0] if df[c].notna().any() else None
                    for c in df.columns],
    })
    return out.sort_values("pct_missing", ascending=False)

profile(raw)
```

```
print(f"rows          : {len(raw):,}")
print(f"exact duplicates: {raw.duplicated().sum():,}")
print(f"duplicate orders: {raw.duplicated(subset='order_id').sum():,}")
print()
print(raw[["quantity", "unit_price"]].describe().T)
```

#### TASK 2.2

How many rows are duplicates by `order_id`?

```
n_duplikat = None

checks.task("2.2", n_duplikat)
```

### 4. Cleaning, in the right order (DEMO + TASK)

**Order matters**, and this is the part students most often get wrong:

1. **Duplicates first** — before anything is counted or averaged.
2. **Types before validity** — the string `"-999"` is not `< 0`.
3. **Derived columns last** — recompute them *after* their inputs are clean, or the old, corrupted values persist.

```
df = raw.copy()
n0 = len(df)

# --- step 1: duplicates -----
df = df.drop_duplicates(subset="order_id", keep="first")
print(f"after dedup: {len(df):,} rows (removed {n0 - len(df):,})")
```

```
# --- step 2: text normalisation -----
print("region, before:", sorted(raw["region"].dropna().unique()))

df["region"] = (df["region"].str.strip().str.title()
               .replace({"": np.nan}))

print("region, after :", sorted(df["region"].dropna().unique()))
print("missing region:", df["region"].isna().sum())
```

```
# --- step 3: validity -----
# The -999 sentinel is already NaN thanks to na_values, but be explicit:
# any non-positive quantity or price is impossible, not merely unusual.
bad_qty = (df["quantity"] <= 0).sum()
bad_price = (df["unit_price"] <= 0).sum()
print(f"impossible quantities: {bad_qty}    impossible prices: {bad_price}")

df.loc[df["quantity"] <= 0, "quantity"] = np.nan
df.loc[df["unit_price"] <= 0, "unit_price"] = np.nan
```

## 5. Missing data: diagnose before you treat (DEMO)

`unit_price` is missing for about 6% of rows. Whether we may simply drop those rows depends on **why** they are missing (Chapter 2, section 2.6.1):

- **MCAR** — missingness unrelated to anything. Deletion loses power but is unbiased.
- **MAR** — missingness depends on *observed* variables. Condition on them.
- **MNAR** — missingness depends on the *unobserved value itself*. Unfixable from the data alone; report the limitation.

You cannot prove MAR from the data, but you can gather evidence: build an indicator and test whether it is associated with what you *did* observe.

```
df["price_missing"] = df["unit_price"].isna()

print("Is missingness associated with the observed variables?\n")
print("by region:")
print((df.groupby("region", observed=True)["price_missing"]
        .agg(["mean", "size"]).rename(columns={"mean": "pct_missing"})
        .assign(pct_missing=lambda d: (d.pct_missing * 100).round(2))))
print("\nby channel:")
print((df.groupby("channel", observed=True)["price_missing"]
        .mean().mul(100).round(2)))
print("\nmean quantity, missing vs observed price:")
print(df.groupby("price_missing")["quantity"].mean().round(3))
```

The rates are flat across region and channel, and mean quantity barely moves. That is consistent with **MCAR** — which here is the truth, because the generator dropped prices at random.

Even so, we impute rather than delete, for two reasons: deletion throws away 300 otherwise-good orders, and a *grouped* median preserves regional price structure that a global median would flatten.

```
before = df["unit_price"].median()

df["unit_price_was_missing"] = df["unit_price"].isna().astype("int8")
df["unit_price"] = df.groupby("region", observed=True)["unit_price"].transform(
    lambda s: s.fillna(s.median()))

print(f"median price before imputation: {before:,.0f}")
print(f"median price after imputation: {df['unit_price'].median():,.0f}")
print(f"still missing: {df['unit_price'].isna().sum()}")
```

The `unit_price_was_missing` indicator is not decoration. If missingness is ever informative — and under MNAR it always is — this column is the only way a model can use that information.

## TASK 2.3

Drop the rows whose `quantity` is still missing, then confirm your cleaned frame still carries the missingness indicator.

```
df_bersih = None          # <- your cleaned DataFrame

# TODO: drop rows where quantity is missing, e.g.
# df_bersih = df.dropna(subset=["quantity"]).copy()

checks.task("2.3", df_bersih)
```

## TASK 2.4

Confirm no impossible quantities survive.

```
checks.task("2.4", df_bersih)
```

## 6. Outliers (DEMO)

Three univariate rules, and they disagree — which is the point.

- **z-score**: assumes normality, and is itself corrupted by the outliers (one extreme value inflates `s` and hides itself).
- **modified z-score**: uses the median and MAD, so it does not break down.
- **IQR rule**: distribution-free, but flags a great many points on skewed data.

```
def flag_outliers(s):
    s = s.dropna()
    z = (s - s.mean()) / s.std(ddof=1)
    mad = (s - s.median()).abs().median()
    mz = 0.6745 * (s - s.median()) / mad if mad > 0 else pd.Series(0.0, index=s.index)
    q1, q3 = s.quantile([0.25, 0.75])
    iqr = q3 - q1
    return pd.DataFrame({
        "zscore": z.abs() > 3,
        "mod_z": mz.abs() > 3.5,
        "iqr": (s < q1 - 1.5 * iqr) | (s > q3 + 1.5 * iqr),
    })

flags = flag_outliers(df["unit_price"])
print(flags.sum().to_frame("n_flagged"))
print(f"\n{len(flags):,} non-missing prices")
print("\nAgreement between rules:")
print(flags.value_counts().head())
```

The plain z-score finds far fewer than the modified z-score — exactly the masking effect described above. The IQR rule flags the most, because `unit_price` is log-normal and a long right tail is *normal for this variable*, not anomalous.

Treatment: these prices are extreme but **possible**, so we winsorise rather than delete, and we keep the original column so the decision stays inspectable.

```
work = df.dropna(subset=["quantity"]).copy()
lo, hi = work["unit_price"].quantile([0.001, 0.999])
work["unit_price_winsor"] = work["unit_price"].clip(lo, hi)

print(f"clipping bounds: {lo:,.0f} .. {hi:,.0f}")
print(f"values clipped : {(work['unit_price'] != work['unit_price_winsor']).sum()}")
print(f"\nmean before: {work['unit_price'].mean():,.0f}")
print(f"mean after : {work['unit_price_winsor'].mean():,.0f}")
```

**Never** delete outliers because they hurt your fit. If you remove points, state which, how many, and by what rule — and fix the rule *before* seeing its effect on the result.

```
# --- derived columns, recomputed AFTER their inputs are clean -----
work["quantity"] = work["quantity"].astype(int)
work["amount"] = work["quantity"] * work["unit_price_winsor"]
work["month"] = work["order_date"].dt.month
work["weekday"] = work["order_date"].dt.day_name()
print(work[["quantity", "unit_price_winsor", "amount"]].describe().T.round(1))
```

## 7. Joining, with the cardinality checked (DEMO)

A join you have not verified is an act of faith. `validate=` raises instead of silently multiplying rows; `indicator=True` tells you what failed to match.

```
customers = pd.read_csv(DATA / "customers.csv", parse_dates=["signup_date"])
print("customers:", customers.shape,
      "\n| unique ids:", customers["customer_id"].nunique())

merged = pd.merge(
    work, customers,
    on="customer_id",
    how="left",
    validate="many_to_one",      # raises if customers has duplicate ids
    indicator=True,             # adds _merge: left_only / both / right_only
)
print("\nrows before join:", len(work), " after:", len(merged))
print(merged["_merge"].value_counts().to_string())
```

```
# Always be able to explain the unmatched rows.
unmatched = merged.loc[merged["_merge"] == "left_only", "customer_id"]
print(f"{len(unmatched)} orders reference a customer absent from customers.csv")
print(f"across {unmatched.nunique()} distinct customer ids")
print("\nSample:", sorted(unmatched.unique())[:8])
```

### TASK 2.5

How many orders failed to match a customer?

```
n_tidak_cocok = None

checks.task("2.5", n_tidak_cocok)
```

### Now break it on purpose (DEMO)

This is what the check is protecting you from.

```
dirty_customers = pd.concat([customers, customers.head(50)], ignore_index=True)

bad_join = pd.merge(work, dirty_customers, on="customer_id", how="left")
print(f"rows before: {len(work):,} after a silent bad join: {len(bad_join):,}")
print(f"inflation : {len(bad_join) / len(work) - 1:+.2%}")

try:
    pd.merge(work, dirty_customers, on="customer_id",
              how="left", validate="many_to_one")
except Exception as exc:
    print(f"\nwith validate=: {type(exc).__name__}")
    print(f" {exc}")
```

Every statistic computed on `bad_join` would be wrong, every standard error too small, and nothing would have warned you.

## 8. Reshaping and aggregating (DEMO)

```
# Group-wise aggregation: the modelling table for a monthly report.
summary = (merged.groupby(["region", "segment"], observed=True)
            .agg(revenue=("amount", "sum"),
                 n_orders=("order_id", "nunique"),
                 avg_basket=("amount", "mean"),
                 first_order=("order_date", "min"))
            .reset_index())
summary["revenue"] = (summary["revenue"] / 1e6).round(1)      # million rupiah
summary.head(10)
```

```
# transform: broadcast a group statistic back onto every row.
# This idiom is the backbone of feature engineering in week 5.
merged["amount_vs_region_mean"] = (
    merged["amount"] / merged.groupby("region", observed=True)["amount"]
    .transform("mean"))
print(merged[["region", "amount", "amount_vs_region_mean"]].head())
```

```
# wide <-> long
wide = merged.pivot_table(index="region", columns="month",
                           values="amount", aggfunc="sum", observed=True)
print("wide:", wide.shape)

long = wide.reset_index().melt(id_vars="region",
                               var_name="month", value_name="amount")
print("long:", long.shape)
long.head()
```

## 9. Saving a model-ready file (DEMO)

Convert once, then work in Parquet: typed, compressed, columnar, and it does not require you to re-specify `parse_dates` and `dtype` on every read.

```

OUT = ROOT / "output"
OUT.mkdir(exist_ok=True)

final = merged.drop(columns=["_merge", "price_missing"])
final.to_parquet(OUT / "sales_processed.parquet", index=False)
final.to_csv(OUT / "sales_processed.csv", index=False)

roundtrip = pd.read_parquet(OUT / "sales_processed.parquet")
print("saved:", final.shape)
print("dtypes survived the round trip:",
      (roundtrip.dtypes == final.dtypes).all())
print("\n", roundtrip.dtypes.to_string())

```

## 10. Take-home (assessed)

**A. The wrong order.** Write an alternative cleaning function that does the steps in a deliberately wrong order: winsorise *before* removing duplicates, and check validity *before* converting types. Report the difference in final row count and mean `amount`, and explain each difference mechanically.

**B. A quality-rule engine.** Write `check_quality(df, rules)` where `rules` maps a column name to a predicate, returning a DataFrame with `column`, `n_violations`, `pct` and three example row indices. Support at least one range rule, one regex rule, and one cross-field rule (a rule that takes the whole frame — e.g. that `amount == quantity * unit_price`).

**C. Entity deduplication.** `customers.csv` is clean. Make it dirty: create a version where the same customer appears as "PT Sumber Rejeki", "PT. Sumber Rejeki", "Sumber Rejeki, PT" and "pt sumber rejeki tbk". Write a normalisation function that maps all four to one key, then explain in two sentences why a purely rule-based approach eventually fails and what technique you would add.

**Submit:** this notebook, executed top to bottom from a fresh kernel, with all five checks green and sections A–C completed.

---

**Next week:** we take `sales_clean.csv` and actually *look* at it.